

Главная модель: actual → описание → проверки

```
import static org.assertj.core.api.Assertions.assertThat;

assertThat(actualPrice)
    .as("Цена заказа %s", orderId)
    .isPositive()
    .isBetween(100, 200)
    .isEqualTo(180);
```

`assertThat(actual)` выбирает assertion по типу actual. Поэтому IDE предлагает разные методы для String, числа, списка, Optional и т. д.

Строки

```
assertThat(orderNumber)
    .isNotBlank()
    .startsWith("order-")
    .endsWith("00042")
    .contains("2026")
    .doesNotContain("error")
    .matches("order-\\d{4}-\\d{5}");
```

Метод	Смысл
<code>isNotNull()</code>	Не null, но пустая строка допустима.
<code>isNotEmpty()</code>	Не null и длина > 0; пробелы допустимы.
<code>isNotBlank()</code>	Есть хотя бы один непробельный символ.

Числа и диапазоны

```
assertThat(age).isGreaterThanOrEqualTo(18);
assertThat(count).isZero();
assertThat(total).isPositive().isLessThan(1_000);
assertThat(percent).isBetween(0, 100);
```

as() ставится до проверки

```
assertThat(user.active())
    .as("Активность пользователя %s", user.name())
    .isTrue();
```

`as` добавляет бизнес-контекст. Expected/actual AssertJ покажет сам. Если поставить `as` после упавшего assertion, выполнение до него не дойдёт.

Null и boolean

```
assertThat(user).isNotNull();
assertThat(deletedUser).isNull();
assertThat(user.active()).isTrue();
assertThat(order.cancelled()).isFalse();
```

Проверяйте значение напрямую. Конструкция `assertThat(x != null).isTrue()` теряет тип и обычно даёт менее полезную диагностику.

Частые ошибки

- `assertThat(expected).isEqualTo(actual)` — поменяны местами `expected` и `actual`.
- `isNotNull()` вместо требуемого `isNotBlank()`.
- Слишком слабый assertion: `contains`, когда контракт требует точное равенство.
- Несколько несвязанных смыслов в одной длинной цепочке.
- Проверка вычисленного `boolean` вместо самого объекта.

Шаблон читаемого теста

```
// given
Order order = service.create(request);

// then
assertThat(order.id()).as("id созданного заказа")
    .isPositive();
assertThat(order.status()).isEqualTo(NEW);
```

Как выбрать проверку коллекции

Метод	Лишние	Порядок
<code>contains(a,b)</code>	можно	учитывает относительный
<code>containsOnly(a,b)</code>	нельзя	не важен
<code>containsExactly(a,b)</code>	нельзя	важен
<code>containsExactlyInAnyOrder</code>	нельзя	не важен

```
assertThat(statuses)
    .containsExactly("NEW", "PAID", "SHIPPED");
```

Optional: сначала presence, затем value

```
assertThat(optionalUser)
    .isPresent()
    .get()
    .extracting(User::name)
    .isEqualTo("Ivan");

assertThat(emptyUser).isEmpty();
assertThat(optionalStatus).contains("READY");
```

Тип цепочки: `Optional<User>` → `User` → `String`. Поэтому после извлечения одного имени нужен `isEqualTo`, а не `containsExactly`.

Размер, элементы и предикаты

```
assertThat(users)
    .isNotEmpty()
    .hasSize(3)
    .contains(ivan)
    .doesNotContain(admin)
    .allMatch(u -> u.age() >= 18)
    .anyMatch(User::active)
    .noneMatch(u -> u.name().isBlank());
```

Не дублируйте контракт без пользы: после `containsExactly(a,b,c)` отдельный `hasSize(3)` обычно избыточен.

Map: ключи, значения и пары

```
assertThat(headers)
    .containsEntry("Content-Type", "application/json")
    .containsKey("X-Request-Id")
    .doesNotContainKey("Authorization");

assertThat(headers).containsOnly(
    entry("A", "1"), entry("B", "2"));
```

Для `entry`: `import static org.assertj.core.api.Assertions.entry;`

Один элемент

```
assertThat(users)
    .singleElement()
    .extracting(User::name)
    .isEqualTo("Ivan");

assertThat(users).first().isEqualTo(ivan);
assertThat(users).last().isEqualTo(petr);
```

Что ослабляет тест

Слабо

```
assertThat(list)
    .isNotEmpty();
```

- Вызов `optional.get()` до assertion даст `NoSuchElementException` вместо понятного отчёта.
- `isEqualTo(List.of(...))` работает, но специализированный `containsExactly` лучше показывает расхождение элементов.

Контракт

```
assertThat(list)
    .containsExactly(a,b);
```

extracting: одно поле списка

```
assertThat(users)
    .extracting(User::name)
    .containsExactly("Ivan", "Anna", "Petr");
```

Было `List<User>`, стало assertion над извлечёнными именами. Method reference типобезопаснее строкового `extracting("name")`.

Несколько полей → tuple

```
import static org.assertj.core.api.Assertions.tuple;

assertThat(users)
    .extracting(User::name, User::age)
    .containsExactly(
        tuple("Ivan", 28),
        tuple("Anna", 31),
        tuple("Petr", 22));
```

Порядок значений в каждом tuple обязан совпадать с порядком функций в `extracting`.

Фильтрация до извлечения

```
assertThat(users)
    .filteredOn(User::active)
    .extracting(User::name, User::age)
    .containsExactly(
        tuple("Ivan", 28),
        tuple("Petr", 22));
```

Читается как контракт: «среди активных пользователей ожидаю именно эти имя/возраст». Без `filteredOn` в результат попадёт и Anna.

Варианты filteredOn

```
assertThat(users)
    .filteredOn(u -> u.age() >= 18)
    .extracting(User::name)
    .contains("Ivan");

assertThat(users)
    .filteredOn("active", true)
    .hasSize(2);
```

Лямбда или method reference предпочтительнее строкового имени свойства: компилятор помогает при рефакторинге.

flatExtracting: расплющить вложенные списки

```
List<Order> orders = List.of(
    new Order(List.of("Book", "Pen")),
    new Order(List.of("Notebook")));

assertThat(orders)
    .flatExtracting(Order::items)
    .containsExactly("Book", "Pen", "Notebook");
```

`extracting(Order::items)` даст список списков. `flatExtracting` объединит элементы в одну последовательность.

Диагностика ошибки компиляции

1. Посмотрите тип сразу после `assertThat(...)`.
2. Проверьте, к какому объекту применяется method reference.
3. Сначала снимите обёртку: `Optional` → `get()`, затем извлекайте поле.
4. Для одного результата используйте `isEqualTo`; для набора — `containsExactly`.

Когда лучше сравнить DTO целиком

Если контракт включает почти все поля, десяток `extracting` ухудшает читаемость. Создайте expected DTO и используйте обычное или recursive comparison. `extracting` особенно хорош для выбранных полей коллекции.

Soft assertions: увидеть все связанные ошибки

```
SoftAssertions.assertSoftly(softly -> {
    softly.assertThat(product.name())
        .as("name").isEqualTo("Keyboard");
    softly.assertThat(product.price())
        .as("price").isEqualToComparingTo("99.90");
    softly.assertThat(product.available())
        .as("available").isTrue();
});
```

Используйте для полей одного объекта/ответа. Не объединяйте несвязанные сценарии: отчёт станет шумным.

Исключения без try/catch

```
assertThatThrownBy(() -> service.findById(-1))
    .isInstanceOf(IllegalArgumentException.class)
    .hasMessage("id must be positive")
    .hasNoCause();

assertThatExceptionOfType(IOException.class)
    .isThrownBy(client::read);

assertThatCode(service::ping)
    .doesNotThrowAnyException();
```

BigDecimal и floating point

```
assertThat(new BigDecimal("10.0"))
    .isEqualToComparingTo("10.00");

assertThat(actualDouble)
    .isCloseTo(10.0, within(0.001));
```

`BigDecimal.equals` учитывает scale; `isEqualToComparingTo` сравнивает числовое значение. Для вычисляемых double задавайте tolerance.

Recursive comparison для DTO

```
assertThat(actual)
    .usingRecursiveComparison()
    .ignoringFields("id", "createdAt")
    .isEqualTo(expected);
```

Сравнивает вложенные поля рекурсивно, даже если DTO не реализует подходящий `equals`.

Полезные настройки

```
.ignoringCollectionOrder()
.withComparatorForType(comparator, BigDecimal.class)
.withEqualsForFields(predicate, "email")
.ignoringActualNullFields()
```

Игнорируйте только технические поля, не входящие в проверяемый контракт. Большой список исключений маскирует дефекты.

Даты и время

```
assertThat(actualInstant)
    .isAfter(start)
    .isBefore(end);

assertThat(actualTime)
    .isCloseTo(expectedTime, within(2, ChronoUnit.SECONDS));
```

Захватите `start` до действия, `end` после. Не сравнивайте серверное время с одной точкой «сейчас» без допустимого окна.

SoftAssertions: опасная форма

```
SoftAssertions softly = new SoftAssertions();
softly.assertThat(a).isEqualTo(expectedA);
softly.assertThat(b).isEqualTo(expectedB);
softly.assertAll(); // обязательно
```

У ручного объекта забытый `assertAll()` делает тест ложно зелёным. Локальный `assertSoftly` безопаснее: финальная проверка вызывается автоматически.

Condition для повторяемого правила

```
Condition<Integer> even = new Condition<>(  
    value -> value % 2 == 0,  
    "even number");  
  
assertThat(10).is(even);  
assertThat(numbers).are(even);
```

Для часто повторяемого доменного языка рассмотрите custom assertion. Он должен выражать смысл: `assertThat(order).isPaid().hasTotal(...)`, а не просто прятать `isEqualTo`.

Другие способы проверок

Инструмент	Когда уместен
JUnit	Простые базовые проверки, <code>assertAll</code> , нет дополнительной библиотеки.
AssertJ	Объекты, коллекции, DTO, fluent-цепочки и богатая диагностика.
Hamcrest	Matcher API; часто inline в REST Assured body().
JSON schema	Структура и типы API-контракта, но не вся бизнес-логика.
Mockito verify	Взаимодействие с зависимостью, только если вызов сам является контрактом.

```
assertEquals(expected, actual); // JUnit  
assertThat(actual, equalTo(expected)); // Hamcrest  
assertThat(actual).isEqualTo(expected); // AssertJ
```

API-тест: Hamcrest + AssertJ

```
Response response = given().when().get("/booking/42");  
response.then().statusCode(200);  
  
Booking actual = response.as(Booking.class);  
assertThat(actual)  
    .usingRecursiveComparison()  
    .isEqualTo(expected);
```

Transport-контракт удобно проверить REST Assured/Hamcrest, а связанный набор полей DTO — AssertJ.

Checklist хорошей проверки

1. Actual находится слева: `assertThat(actual)`.
2. Assertion соответствует типу данных.
3. Строгость совпадает с контрактом: состав, порядок, дубликаты.
4. `as` объясняет бизнес-контекст и стоит до проверки.
5. Optional проверен до извлечения значения.
6. Для `double` задан `tolerance`; для денег осознанно обработан `scale`.
7. Soft assertions объединяют только связанные свойства.
8. Recursive comparison не игнорирует бизнес-поля.
9. Падение сообщает, что ожидали и что получили.
10. Проверяется результат, а не детали реализации.

Антипаттерны

- `assertThat(true).isTrue()` после сложной ручной логики.
- Цикл с assertion, когда достаточно `extracting/allMatch`.
- Java assert: без JVM-флага `-ea` он отключён.
- Проверка `toString()` вместо полей объекта.
- Только `isNotNull` для ответа, у которого есть бизнес-контракт.
- Проверка каждого внутреннего вызова mock-объекта вместо результата.

Алгоритм при падении

1. Прочитайте `description`, `expected` и `actual`.
2. Проверьте тестовые данные и тип `actual`.
3. Убедитесь, что assertion выражает правильный контракт.
4. Сравните минимальное различающее поле.
5. Классифицируйте причину: продукт, тест, данные или окружение.

Что значит уровень middle+

Вы не просто получаете зелёный тест, а можете объяснить выбранную строгость, сделать падение диагностируемым, не замаскировать дефект игнорированием полей и выбрать подходящий инструмент для каждого уровня контракта.